



INSTITUTO FEDERAL DA BAHIA
DIRETORIA ACADÊMICA
CURSO TECNOLÓGICO DE ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS

PEDRO LUCAS CARDOSO GOVEIA

ESTRATÉGIAS DE SINCRONIZAÇÃO DE DADOS EM SISTEMAS
OFFLINE-FIRST E PEER-TO-PEER

EUNÁPOLIS - BA

2025

PEDRO LUCAS CARDOSO GOVEIA

**ESTRATÉGIAS DE SINCRONIZAÇÃO DE DADOS EM SISTEMAS
OFFLINE-FIRST E PEER-TO-PEER**

Trabalho de conclusão de curso apresentado como requisito parcial à obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas da Coordenação do Curso de Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia da Bahia.

Orientador: Jurandir da Cruz Barbosa

EUNÁPOLIS - BA

2025

AGRADECIMENTOS

RESUMO

Com o desenvolvimento da computação móvel, projetos são pensados com foco nas qualidades deste tipo de sistema. Em contraste com aplicações web, o estado offline não é um erro ou algo temporário, mas fato que o desenvolvimento móvel deve relevar. Assim o paradigma Offline-First se consolida como uma realidade; este paradigma descreve, dentre outros aspectos, a necessidade de sincronização de dados dos dispositivos que se conectam ao sistema, para manter a consistência e disponibilidade desses dados. Em cenários onde há centralização dos processos, como em um servidor web, existem estratégias específicas para a manutenção dos arquivos e estados; e o mesmo se aplica à computação descentralizada. Redes Peer-to-Peer são uma boa alternativa a um servidor devido o seu baixo custo e privacidade, atualmente ganharam mais tração com o surgimento das blockchains. No entanto, redes Peer-to-Peer apresentam seus próprios desafios de sincronização: peer churn, consistência entre as réplicas e eficiência na sincronização de dados. Desse modo, a necessidade do estudo de estratégias de sincronização em redes Peer-to-Peer em sistemas Offline-First é importante para a reflexão e o direcionamento na escolha da implementação da sincronização de dados congruente ao sistema em produção. Este estudo deve ser realizado com o uso de métricas em ambiente controlado com nós funcionando em rede com conexão descentralizada para obter dados aptos a serem analisados de modo a serem dispostos a comparação das vantagens, limitações e comportamentos dos diferentes tipos de sincronização. No mesmo sentido, o desenvolvimento de uma aplicação Offline-First real para proporcionar substâncias a cada uma dessas aplicações é indispensável para validar as discussões realizadas sobre cada estratégia de sincronização de dados.

Palavras-chave: Offline-First, Peer-to-Peer, Sincronização de dados, CRDT, CAP.

ABSTRACT

With the development of mobile computing, projects are increasingly designed with a focus on the qualities of this type of system. In contrast to web applications, the offline state is not considered an error or a temporary condition, but rather a fact that mobile development must take into account. Thus, the Offline-First paradigm has been consolidated as a reality. This paradigm describes, among other aspects, the need for data synchronization across devices connected to the system, in order to maintain the consistency and availability of such data. In scenarios where processes are centralized, such as on a web server, there are specific strategies for maintaining files and states; the same applies to decentralized computing. Peer-to-Peer networks are a good alternative to servers due to their low cost and privacy, and they have gained further traction with the rise of blockchains. However, Peer-to-Peer networks present their own synchronization challenges: peer churn, replica consistency, and synchronization efficiency. Therefore, studying synchronization strategies in Peer-to-Peer networks within Offline-First systems is essential to guide decision-making in selecting data synchronization implementations that are congruent with the system in production. This study must be carried out using metrics in a controlled environment with nodes operating in a decentralized network, in order to collect data suitable for analysis and comparison of the advantages, limitations, and behaviors of different synchronization strategies. In the same sense, the development of a real Offline-First application to provide substance to each of these strategies is indispensable for validating the discussions conducted about each synchronization approach.

Keywords: Offline-First, Peer-to-Peer, Data Synchronization, CRDT, CAP.

LISTA DE ILUSTRAÇÕES

LISTA DE TABELAS

LISTA DE SIGLAS E ACRÔNIMOS

1. ADS – *Análise e Desenvolvimento de Sistemas*
2. CAP – *Consistency, Availability, Partition Tolerance* – Consistência, Disponibilidade e Tolerância à Partição
3. CmRDT – *Commutative Replicated Data Type* – Tipo de Dado Replicado Baseado em Operações
4. CRDT – *Conflict-free Replicated Data Type* – Tipo de Dado Replicado Sem Conflito
5. CvRDT – *Convergent Replicated Data Type* – Tipo de Dado Replicado Baseado em Estado
6. HTTP – *HyperText Transfer Protocol* – Protocolo de Transferência de Hipertexto
7. IPFS – *InterPlanetary File System* – Sistema de Arquivos Interplanetário
8. OT – *Operational Transformation* – Transformação Operacional
9. P2P – *Peer-to-Peer* – Rede de Pares
10. RAID – *Redundant Array of Independent Disks* – Conjunto Redundante de Discos Independentes
11. RFC – *Request for Comments*
– Solicitação de Comentários (Documentos de Padronização)
12. SQL – *Structured Query Language*
– Linguagem de Consulta Estruturada
13. TCC – *Trabalho de Conclusão de Curso*

14. TCP/IP – *Transmission Control Protocol / Internet Protocol* – Protocolo de Controle de Transmissão / Protocolo de Internet

SUMÁRIO

1	INTRODUÇÃO	1
1.1	PROMLEMÁTICA	2
1.2	OBJETIVOS	2
1.3	JUSTIFICATIVA	3
1.4	ESTRUTURA DO TRABALHO	4
2	METODOLOGIA	4
2.1	ACERCA DAS CATEGORIAS DA PESQUISA	4
2.2	ACERDA DO MÉTODO CIENTÍFICO UTILIZADO	5
2.3	ACERCA DO DESENVOLVIMENTO DA APLICAÇÃO	6
3	REFERÊNCIAL TEÓRICO	7
3.1	LINGUAGENS DE PROGRAMAÇÃO	7
3.2	MODELO CLIENTE-SERVIDOR	7
3.3	CONCEITOS ESSENCIAIS PARA A CONSTRUÇÃO DA APLICAÇÃO MÓVEL	8
3.3.1	SISTEMAS <i>OFFLINE-FIRST</i>	8
3.3.2	LINGUAGEM DE PROGRAMAÇÃO: <i>KOTLIN</i>	9
3.3.3	BANCO DE DADOS: <i>ROOM</i>	9
3.4	CONCEITOS ESSENCIAIS DO <i>MIDDLEWARE</i> DE SINCRONIZAÇÃO	10
3.4.1	REDES <i>PEER-TO-PEER</i>	10
3.4.2	LINGUAGEM DE PROGRAMAÇÃO: <i>GOLANG</i>	10
3.4.3	AMBIENTE DE TESTES: <i>DOCKER</i>	11
3.5	ESTRATÉGIAS DE SINCRONIZAÇÃO	11
3.5.1	TEOREMA <i>CAP</i>	12
3.5.2	ESTRATÉGIA DE SINCRONIZAÇÃO: <i>CRDT</i>	12
3.5.3	CONSISTÊNCIA EVENTUAL FORTE (<i>SEC</i>)	13
3.5.4	VETORES DE VERSÃO	14
3.6	TRABALHOS RELACIONADOS	15

1 INTRODUÇÃO

Sistemas de *software* referenciam a infraestrutura disponibilizada pelo ambiente virtual da máquina de executar aplicações; desse modo, esses sistemas incluem o sistema operacional e outros *softwares* utilitários que oferecem serviços fundamentais (TANENBAUM; BOS, 2015). Aplicações, portanto, são *softwares* que agregam funcionalidades adicionais a dispositivos (computadores, celulares, *tablets*, etc.) que o usuário necessita. Assim, Tanenbaum e Van Steen elucidam sobre o uso e a natureza de sistemas distribuídos:

Os usuários (sejam pessoas ou programas) acreditam que estão lidando com um único sistema. Isso significa que, de uma forma ou de outra, os componentes autônomos precisam colaborar. A maneira de estabelecer essa colaboração está no cerne do desenvolvimento de sistemas distribuídos. (TANENBAUM; STEEN, 2007, p. 2)

Estratégias de sincronização de dados são utilizadas em muitas aplicações, devido à necessidade de intercomunicação entre elas, como nas aplicações *web* descritas por Tanenbaum e Van Steen (TANENBAUM; STEEN, 2007). Desse modo, a exploração das soluções existentes, tanto do ponto de vista do uso dos recursos quanto do desenvolvimento de novas ferramentas é uma tarefa desafiadora para os desenvolvedores e útil para o mercado atual. Empresas que atuam na área rural, que utilizam equipamentos com contato intermitente com alguma rede (*internet* ou *intranet*) podem se beneficiar com uso de tecnologias com o perfil *offline*. E esse paradigma de desenvolvimento requer um planejamento sobre a aplicação da sincronização.

Logo, para esse trabalho será aplicada a construção de sistema no paradigma *Offline-First* dentro de uma rede *Peer-to-Peer*, devido aos desafios encontrados nesse tipo de rede para a sincronização das réplicas, assim criando uma rede *Device-to-Device (D2D)*, onde os dispositivos se comunicam entre si de maneira independente. Não será o foco deste trabalho, no entanto, a descrição da instalação da rede e a discussão da construção da aplicação, mas sim a exploração das diferentes estratégias de sincronização e seu comportamento no contexto *Peer-to-Peer* de uma aplicação móvel.

1.1 PROMLEMÁTICA

Com a crescente difusão da computação móvel, sistemas devem levar em consideração o funcionamento *offline*, isso implica em estratégias para lidar com conectividade intermitente (FEYERKE, 2021); esse problema se intensifica em redes *Peer-to-Peer* pois não há servidor central para coordenar a distribuição de dados. Conforme o teorema *CAP* e o resultado de impossibilidade *FLP* um sistema deve realizar compromissos em relação a consistência ou disponibilidade no seu projeto, pois não é possível possuir esses dois fatores e tolerância a partição (essencial em sistemas distribuídos) (GILBERT; LYNCH, 2012), pois não há algoritmo determinístico capaz de encontrar consenso entre as réplicas em um contexto que assume falhas (FISCHER; LYNCH; PATERSON, 1985). Assim, o objetivo é manter a consistência total de dados entre as réplicas.

As estratégias de sincronização possibilitam a construção de sistemas pensados para o funcionamento *offline*, garantindo a consistência eventual entre as réplicas. Elas atendem a problemática de como sincronizar dados de forma eficiente e consistente. Assim, esse texto visa compreender os comportamentos, vantagens e limitações quando aplicadas a ambientes descentralizados. Os seguintes trabalhos realizam comparativos de modelos de consistência de dados: *Conflict-Free Replicated Data Types* (SHAPIRO et al., 2015), compara *CRDTs* com *OT*; (VIOTTI; VUKOLIĆ, 2016) apresenta um estudo comparativo de modelos de consistência em sistemas distribuídos. No entanto, observa-se uma lacuna na literatura quanto a um estudo mais generalista e sistemático que compare, de forma abrangente, estratégias diversas de sincronização sob métricas e cenários comuns, particularmente em ambientes descentralizados.

1.2 OBJETIVOS

O objetivo geral deste trabalho é desenvolver um sistema para investigar e analisar estratégias de sincronização de dados em redes *P2P* usando métricas para destacar limitações, vantagens e comportamentos em cenários onde existe conectividade intermitente, propondo reflexões e direcionamentos na aplicação das estratégias utilizadas, criando um protótipo funcional de uma aplicação que testa a investigação teórica.

Portanto, os objetivos específicos deste trabalho são:

- Analisar estratégias de sincronização sem considerar a topologia das redes onde são aplicadas;
- Desenvolver um protótipo funcional do sistema com componente de sincronização reutilizável e aplicação móvel para teste;
- Testar o componente de sincronização realizando a discussão sobre a performance com base nas métricas com os dados obtidos.

1.3 JUSTIFICATIVA

Atualmente, com o aumento da computação móvel, existe uma crescente necessidade de sistemas que melhor se adaptam a cenários que não possuem conectividade sem falhas. Sistemas com o paradigma *Offline-First* (construídos de maneira a levar em consideração o funcionamento sem o uso da rede) estão se tornando cada vez mais comuns, pois *Offline-First* não é um erro mas um fato da computação móvel (FEYERKE, 2021). Logo, esse modo de pensar sistemas introduz o desafio da sincronização de dados.

Assim, o estudo de estratégias de sincronização de dados em sistemas *Offline-First* em redes *Peer-to-Peer* oferece oportunidades de exploração de técnicas para solucionar problemas onde: a rede *P2P* monta um ambiente de conexão efêmera, os pares são desconectados e reconectados com frequência e a propagação dos dados é realizada de par-a-par; O sistema projetados com o paradigma *Offline-First* leva em consideração, desde sua criação, dois pontos importantes: a possibilidade de desconexão da rede e a necessidade de sincronização de dados quando houver a eventual reconexão com a rede.

Esse tipo de aplicação pode ser usada em empresas que trabalham em campo: em atividades florestais, agricultura, avicultura, dentre outras. A utilização de redes *P2P* é interessante pois pode baixar o custo da infraestrutura e, dependendo das opções de *frameworks* adotados, oferecem maior caráter de privacidade para as operações realizadas. Além disso, oferecem os desafios da sincronização para manter a consistência e a disponibilidade dos dados entre as réplicas. Portanto, a experimentação com o desenvolvimento de uma aplicação móvel real nesse contexto oferece oportunidades para testes e reflexões sobre as possibilidades e os cenários que as estratégias melhor se aplicam.

1.4 ESTRUTURA DO TRABALHO

Este trabalho é organizado em introdução, metodologia, referencial teórico, discussão e considerações finais. A introdução apresenta a ideia geral do estudo, incluindo o objetivo geral, os objetivos específicos e a justificativa da pesquisa. A metodologia resume as técnicas, recursos e métricas utilizadas para o desenvolvimento do componente de sincronização (*middleware*, *software* que opera entre componentes comunicantes de um sistema). O referencial teórico contém toda a informação necessária para a compreensão da discussão realizada: conceitos essenciais de sincronização de dados em sistemas distribuídos, conceituação de redes *Peer-to-Peer*, introdução de tecnologias-chave para o desenvolvimento (Go, Kotlin, Docker e libp2p). A discussão consiste na análise dos dados contextualizados pelas métricas destacadas na seção de metodologia em busca de direcionamentos na aplicação das estratégias de sincronização em redes *Peer-to-Peer*. As considerações finais buscam sintetizar o trabalho e propor formas de expandir a discussão.

2 METODOLOGIA

Esta seção apresenta a metodologia usada para o desenvolvimento deste trabalho, isso inclui: a descrição do estudo realizado, o método de construção do trabalho, assim como as métricas utilizadas para a discussão dos resultados dos testes no produto construído.

2.1 ACERCA DAS CATEGORIAS DA PESQUISA

As pesquisas podem possuir diversas categorias na metodologia científica, algumas delas são: quanto a finalidade, quanto a objetivos, quanto a procedimentos e quanto a natureza da abordagem. Desse modo, podemos definir exemplos dessas categorias relevantes para este texto. Logo, quanto a finalidade há a pesquisa aplicada que possui foco na aplicação, utilização e consequências práticas do conhecimento (GIL, 2008). Quanto aos objetivos destaca-se a pesquisa descritiva que tem como objetivo primordial a descrição das características de determinada população ou fenômeno (GIL, 2002). Quanto aos procedimentos: a pesquisa experimental, determina um objeto de estudo e variáveis que são capazes de influenciá-lo e define formas de controle e observação dos efeitos que a manipulação dessas variáveis possuem no objeto. E, quanto a natureza da abordagem é de destaque a qualitativa onde a análise é sistemática e compreensiva, usada usualmente em estudos de caso e pesquisa-ação, pois não há fórmula para

orientar o pesquisador, assim também precisa ser maleável (GIL, 2008).

Assim o trabalho se encaixa nas categorias: pesquisa aplicada porque visa a produção de direcionamentos para a implementação das estratégias de sincronização; na descritiva quanto aos seus objetivos de observar os fenômenos (as diferentes estratégias de sincronização em contextos distintos) sem interferir; e, experimental quanto aos procedimentos aplicados, isso é, manipulação dos parâmetros de teste para a simulação de contextos diversos com a coleta de dados qualitativos (GIL, 2008).

2.2 ACERDA DO MÉTODO CIENTÍFICO UTILIZADO

O método científico da pesquisa esclarece os procedimentos lógicos que foram usados na pesquisa, assim, o método escolhido foi o hipotético-dedutivo; este foi escolhido pois define uma abordagem experimental para explicar um fenômeno. Isso se alinha com o objetivo do trabalho de coletar métricas de testes sobre um produto desenvolvido para alcançar um conjunto de postulados que governam os fenômenos analisados na discussão (GIL, 2008). As métricas utilizadas para comparação entre as diferentes estratégias estão nas categorias: acurácia de sincronização, latência e propagação, resiliência da rede e adaptabilidade, eficiência de transmissão de mensagem e utilização de recursos (memória e processamento). Assim as métricas consistem:

- Porcentagem de consistência de dados: porcentagem de *nodes* (ou nós, máquinas na rede que consomem e produzem mensagens) que possuem dados no mesmo estado após um período definido de tempo;
- Porcentagem de resolução de conflitos: a porcentagem em que os conflitos são resolvidos corretamente;
- Latência de sincronização: tempo que a mudança em um node leva para alcançar os demais;
- Tolerância de *peer churn* (conexão e desconexão de nodes na rede): degradação de performance com a conexão e desconexão de nodes frequentes;

- Tempo de recuperação: tempo que leva para ressincronizar após falhas;
- Excedente de mensagem: razão de mensagens de sincronização para dados atualizados;
- Uso de banda larga por *node*: quantidade de banda larga usada por *node* por sincronização;
- Utilização de memória: uso de memória em *buffers* e metadados;
- Uso de processamento: processamento de mensagens e resolução de conflitos.

Essas métricas vão ser acessadas através de testes realizados em uma rede, simulando um contexto *Peer-to-Peer*. Assim, para o ambiente de simulação de *nodes* cabe observar os seguintes casos para a coleta de dados: atualização de dados, resolução de conflitos, publicar ou convergir estados (em busca de consistência); Tais observações testam a capacidade do sistema de manter a integridade do fluxo e manter a latência dentro dos limites aceitáveis. Em cenários reais há situações que levam a perda de pacotes, exemplos desses sendo: alta latência e *peer churn* (NASRALLAH et al., 2018); assim, na construção dos testes foi implementada aleatoriedade do contexto da rede para simular cenários reais, juntamente com o agrupamento de *nodes* em subredes.

2.3 ACERCA DO DESENVOLVIMENTO DA APLICAÇÃO

A aplicação de teste foi desenvolvida usando *GoLang* para os *middlewares* de sincronização, *Kotlin* com a plataforma *Android Studio* para a aplicação móvel; para editor de código genérico foi usado o *Neovim*. O ambiente de testes consiste no *Docker* com configurações para auxiliar na simulação de *peer churn*; o sistema operacional *host* é um Debian 12. Para o banco de dados local da aplicação móvel foi usado o *Room* responsável por adicionar uma camada de abstração sobre o *SQLite*.

3 REFERÊNCIAL TEÓRICO

Esta seção apresenta conceitos necessários para o desenvolvimento do trabalho, sendo esses: definição de conceitos básicos (linguagens de programação e modelo cliente-servidor) descrição das tecnologias usadas para a construção da aplicação (*Go*, *Kotlin*, redes *Peer-to-Peer*), diferentes tipos de estratégias de sincronização e tópicos essenciais para o seu entendimento tão como comparações com trabalhos relacionados.

3.1 LINGUAGENS DE PROGRAMAÇÃO

A linguagem nativa de computadores é a binária (uns e zeros), todas as instruções e dados devem ser providas nesse formato para serem processados. Código binário nativo é chamado de linguagem de máquina. Com o passar do tempo foram aparecendo novas formas de construir programas; sucedendo o código de máquina os programas começaram a serem escritos em sequências de números hexadecimais. Em seguida o surgimento linguagens *assembly*, os códigos então começaram a ser escritos usando uma série de comandos mnemônicos que representam um conjunto de instruções em código de máquina. Essas linguagens de programação não são apropriadas para o desenvolvimento de aplicação em larga escala, portanto, o surgimento das linguagens de alto nível como C. Estas definem um conjunto de mecanismos que servem como abstrações das instruções de máquina (*loops*, condicionais, funções, etc) (BAILEY, 2005).

Construir estruturas de dados e algoritmos requer uma linguagem de programação de alto nível (GOODRICH; TAMASSIA; MOUNT, 2011). Logo, para o desenvolvimento da aplicação móvel e dos *middlewares* de sincronização foram, portanto, escolhidas as linguagens de programação *Kotlin* e *Golang*, respectivamente.

3.2 MODELO CLIENTE-SERVIDOR

A escolha dos componentes de um *software*, as interações entre eles e onde atuam, constitui a arquitetura de sistema. Na arquitetura de sistemas categoriza-se, basicamente, os sistemas em centralizados e descentralizados, essas categorizações se baseiam na natureza da relação cliente-servidor; Quando discutindo arquitetura de sistemas o modelo cliente-servidor é essencial. Desse modo, esse modelo define clientes que requisitam serviços de servidores,

onde em sistemas centralizados é definido por: uma máquina cliente que implementa a parte do sistema voltada para o usuário; servidores contém o resto da aplicação (processamento e a camada de dados) (TANENBAUM; STEEN, 2007).

Em sistemas descentralizados essa organização difere, onde em sistemas centralizados a distribuição é alcançada, principalmente, por separar componentes logicamente em diferentes máquinas (distribuição vertical), nos descentralizados a distribuição é alcançada horizontalmente, isso é, clientes ou servidores podem ser divididos em partes equivalentes, cada uma responsável por processar sua parte do trabalho. Uma classe de arquitetura de sistema que implementa distribuição horizontal é *peer-to-peer* (TANENBAUM; STEEN, 2007).

3.3 CONCEITOS ESSENCIAIS PARA A CONSTRUÇÃO DA APLICAÇÃO MÓVEL

A aplicação consiste em um leitor de *ePub* com a possibilidade de fazer anotações, a sincronização será feita a partir dos compartilhamento dessas anotações na rede, todos os pares deveram ter as anotações dos demais. Desse modo, as tecnologias relevantes são: *Golang* e *Kotlin* para linguagens de programação e *Room* para banco de dados local, onde os dados serão armazenados quando no dispositivo celular do usuário.

3.3.1 SISTEMAS OFFLINE-FIRST

Offline-first é um paradigma de programação que especifica, dentre outros aspectos, o comportamento de aplicações em relação a falta de conectividade (OFFLINE FIRST COMMUNITY, n.d.). Um bom exemplo desse paradigma são os *Progressive Web Apps (PWAs)*, para uma aplicação ser um *PWA* esta deve englobar conceitos que formulam o modo que foi construída; dentre eles destaca-se, para esse trabalho, a independência de conexão (BIØRN-HANSEN; MAJCHRZAK; GRØNLI, 2018). A arquitetura define, também, o armazenamento local de dados como componente essencial para sua implementação, e com ele a sincronização e a resolução de conflitos. *Offline-first* assume a existência da necessidade de conexão com alguma rede, sendo essa efêmera, e especifica como aplicações devem funcionar nesse contexto (GUDA, 2025).

A oferta variável de recursos da computação móvel sugere que aplicações para esse

contexto devem ser adaptáveis. Historicamente, adaptação é a troca do recurso por outro ou pela qualidade do serviço prestado, isso é: técnicas de redução, filtragem e transformação de dados para trafegarem na rede; o processo de adaptabilidade é disparado automaticamente após a leitura do contexto. Contexto é toda informação relevante para aplicação que define seu comportamento; desse modo, as informações de contexto podem ser categorizadas em: físicas, relativas ao *hardware*; lógicas, relativas ao *software*; e, sociais, relativas à comunidade de usuários e seu contexto (AUGUSTIN et al., 2001). A necessidade por uma funcionalidade *offline* robusta persiste, como em sistemas de *chat* que funcionam em *intranets* de ambientes isolados (GUDA, 2025), por exemplo, em áreas rurais de plantio, eventos em locais remotos e navios de cruzeiro.

3.3.2 LINGUAGEM DE PROGRAMAÇÃO: *KOTLIN*

Para o desenvolvimento da aplicação móvel, sua construção foi realizada com o *software Android Studio* usando a linguagem de programação *Kotlin*. No desenvolvimento de uma aplicação é importante a escolha de uma plataforma, a plataforma escolhida para a aplicação foi o *Android*. Este é um sistema operacional para dispositivos móveis, sendo usado em *tablets*, celulares, dentre outros. *Android Studio* é o ambiente de desenvolvimento oficial de aplicações para a *Android*, pode ser usado com *Java* e *Kotlin*. *Kotlin* é uma linguagem de programação pragmática que combina os paradigmas de Orientação a Objetos e Programação Funcional com foco em interoperabilidade; a interoperabilidade se dá pela qualidade da linguagem de usar códigos que foram escritos em *Java* em códigos escritos em *Kotlin* (RESENDE, 2020).

Orientação a Objetos é uma paradigma de programação que determina a organização do código em objetos. Um objeto é gerado de uma classe, essa é a especificação de atributos e métodos que o objeto contém em sua definição; cada classe apresenta uma interface consistente para o restante do código sem entregar detalhes inapropriados (GOODRICH; TAMASSIA; MOUNT, 2011). *Kotlin* como uma linguagem orientada a objetos adere a esse paradigma.

3.3.3 BANCO DE DADOS: *ROOM*

Banco de dados são usados em diversos contextos (comercial, científico, pessoal) devido a necessidade do armazenamento persistente de dados; banco de dados são poderosos devido aos seus SGBDs (sistemas de gerenciamento de banco de dados) que provê as funcionalidades de persistência de dados, interface de modificação com uma linguagem de *query*

(SQL, por exemplo), estas são linguagens de modificação de dados e da estrutura do banco de dados; e, gerenciamento de transação (GARCIA-MOLINA; ULLMAN; WIDOM, 2002). Na aplicação móvel, o banco local será onde os dados sincronizados residiram.

Room é uma biblioteca de persistência que funciona sobre *SQLite*, fornecendo abstrações para permitir um acesso mais robusto ao banco de dados local (ANDROID DEVELOPERS, 2025). *SQLite* é um dos maiores sistemas de gerenciamento de banco de dados, estando presente em celulares, computadores, navegadores, televisores, etc; o motivo de seu sucesso pode ser creditado a: sua portabilidade para múltiplas plataformas, ele é compacto e auto-contido, sua confiabilidade e velocidade (GAFFNEY et al., 2022).

3.4 CONCEITOS ESSENCIAIS DO *MIDDLEWARE* DE SINCRONIZAÇÃO

Middleware forma uma camada entre aplicações e plataformas distribuídas para prover transparência de distribuição, isso é, esconder uma porção dos dados, processamento e controle das aplicações (TANENBAUM; STEEN, 2007). Desse modo, será apresentada nesta seção os conceitos necessários para montar essa camada intermediária tão como componentes dos testes.

3.4.1 REDES *PEER-TO-PEER*

Redes *peer-to-peer* são redes onde cada *node* atua como cliente e servidor. Essas redes podem ser de arquitetura estruturada e não estruturada. Nas estruturadas a rede é construída usando um procedimento determinista; um exemplo seria manter uma tabela com identificadores para todos os *nodes*, este mesmo identificador atua para categorizar os dados, assim quando um item é acessado na rede, é retornado o identificador do *node* responsável para ele. Em uma rede não estruturada os *nodes* conhecem uns aos outros de maneira quase aleatória, isso é, cada um mantém uma lista de vizinhos construída de maneira não determinista; a transmissão de dados ocorre da mesma forma (TANENBAUM; STEEN, 2007).

3.4.2 LINGUAGEM DE PROGRAMAÇÃO: *GOLANG*

Golang é uma linguagem de programação simimilar a linguagem C, dela herdou a sintaxe de expressões, tipos de dados básicos e a ênfase em programas que compilam para código de máquina eficiente. Criada por Robert Griesemer, Rob Pike, e Ken Thompson, todos

da *Google*, em 2007 para resolver o problema crescente da complexidade dos sistema da empresa. *Go* é apropriado para a construção de servidores que atuam como infraestutura em rede (DONOVAN; KERNIGHAN, 2016), portanto a escolha para a construção do *middleware*.

A linguagem coloca muita ênfase na eficiência, *Go* possui funcionalidades inspiradas em outras linguagens, como: *garbage collection* de linguagens como *Java*, o conceito de pacotes de *Modula-2*, a sintaxe de métodos de *Oberon-2*, dentre outras funcionalidades. As *goroutines*, fluxos leves de execução, são outro aspecto da língua que demonstra a preocupação por eficiência, onde: criar uma *goroutine* é barato e criar milhões é prático (DONOVAN; KERNIGHAN, 2016).

Alguns exemplos de produtos que usam *Go* são: *Traefik*, um *proxy* reverso e balanceador de carga, isso é, simplifica o roteamento entre vários serviços (microserviços e aplicações em *Docker*) (TRAEFIK LABS, 2025); *Caddy*, plataforma para servir *sites*, aplicações e serviços (CADDY SERVER, 2025); *KrakenD*, simplifica acesso à mutiplos serviços de *backend*, ele faz isso por centralizar, acelerar e proteger as conexões (KRAKEND, 2025).

3.4.3 AMBIENTE DE TESTES: *DOCKER*

Docker é uma plataforma aberta para desenvolvimento, publicação e execução de aplicações. Ele oferta recursos para empacotar e executar uma aplicação em ambiente com isolamento fraco, isso é, um contêiner. *Docker* também oferece ferramentas para orquestração de contêineres, onde é possível manipular a criação, destruição desses; também é possível isolar ambientes, simulando redes distintas. *Docker* é usado a partir da manipulação de seus objetos: contêineres, *networks*, volumes, imagens, etc. Cada objeto atua sobre um sistema operacional definido no contêiner, por exemplo, *Debian*, *Ubuntu*, *Fedora*, dentre outros (DOCKER, INC., 2025).

3.5 ESTRATÉGIAS DE SINCRONIZAÇÃO

As formas de classificar as estratégias de sincronização podem ser de acordo com a necessidade da aplicação em relação a consistência entre as réplicas (TANENBAUM; STEEN, 2007). Assim, será destacado definições de conceitos essenciais para a categorização e implementação das estratégias de sincronização de dados.

3.5.1 TEOREMA CAP

Teorema *CAP* define um *tradeoff* entre consistência e disponibilidade em sistemas distribuídos não confiáveis. Apresentado por Eric Brewer através de um exemplo generalista de serviço *web* para demonstrar essa troca entre disponibilidade, consistência e tolerância a partição. Consistência sendo a capacidade do serviço de retornar a resposta correta para uma requisição; Disponibilidade é a garantia de toda requisição receber uma resposta, mesmo que eventualmente; e, tolerância a partição, este é mais sobre a infraestrutura onde o serviço é executado, portanto, é a capacidade serviço de permanecer funcionando mesmo no caso de perda de comunicação (GILBERT; LYNCH, 2012).

Pode se destacar ainda o resultado de impossibilidade *FLP*; este afirma que em um sistema completamente assíncrono, isso é, em um sistema onde não há garantia de quando uma mensagem será entregue, não existe algoritmo determinístico capaz de encontrar consenso desde que o sistema assuma pelo menos uma falha de execução. Desse modo, entendendo o contexto com conectividade intermitente, é necessário, na construção de sistemas reais, relaxar a assincronicidade (FISCHER; LYNCH; PATERSON, 1985), por exemplo, ao aplicar mecanismos como: consistência eventual forte (GOMES et al., 2017) ou *CRDTs*.

Assim, na construção da aplicação casos onde ocorrem desconexão de par devem ser resolvidos com aplicação de estratégias para manter os dados estruturados e coesos.

3.5.2 ESTRATÉGIA DE SINCRONIZAÇÃO: *CRDT*

Sincronização de dados em sistemas é uma tarefa essencial no estabelecimento de uma equivalência entre duas ou mais coleções de dados. Do mesmo modo, Tanenbaum (TANENBAUM; STEEN, 2007) afirma que o principal desafio na replicação de dados (isso é, manter diferentes cópias de dados em servidores ou localidades diferentes) é assegurar a consistência das réplicas.

Estratégias de sincronização estão presentes tanto em arquiteturas centralizadas e descentralizadas. Inicialmente, há os *CRDTs* (tipos de dado livre de conflito): algoritmos distribuídos que computam o estado de uma aplicação colaborativa do seu histórico de operações. Esse estado deve ser o mesmo para todos os usuários e ser computável sem a necessidade de ter

um componente que centraliza as informações; um servidor central, por exemplo. Possui dois tipos distintos dependendo no que se baseiam: operações ou estados (WEIDNER, 2023).

CRDTs é um sistema replicado, e como tal precisa propagar atualizações para todas as replicas, há dois caminhos para alcançar esse objetivo: replicação baseada em estado e replicação baseada em operação. No primeiro, a estrutura de dados faz uso de uma função de união que integra o estado da replica remota na local; na segunda forma de replicação, a convergência dos estados ocorre através da propagação de operações entre replicas, portanto uma *CRDT* dessa categoria deve gerar, para cada operação, um gerador e um efetor; o primeiro responsável por gerar o segundo ao fim de uma operação, este é então executado nas outras replicas para convergir os estados (PREGUICA; BAQUERO; SHAPIRO, 2018).

CRDTs podem ser entendidas como resultado da evolução, até o momento, de estratégias de sincronização em sistemas distribuídos, a aplicação usando o *CRDT* de operação é o mais apropriado ao sistema de *Offline-First* produzido devido a capacidade das alterações em anotações serem traduzidas em operações (inserir e deletar).

Uma outra forma de sincronização que também faz uso do compartilhamento de estado é a transformação operacional (*OT*). Esta é voltada para arquiteturas centralizadas e é muito usada em editores em grupo; ela resolve problemas similares a *CRDTs*: consistência na manutenção de documentos compartilhados com tempo de resposta curto e edição paralela em ambientes distribuídos, por exemplo (SUN; ELLIS, 1998). Essa estratégia de sincronização é pesquisada desde o fim dos anos 80 e sua importância é inegável, desde artigos, videos, criação de jogos, comunicação *online* em geral. Os desenvolvimentos teóricos e provas experimentais de Ellis et al. que a originaram; atualmente, junto a *CRDT* é um dos métodos mais bem aceitos para resolver o problema de consistência de dados em arquiteturas distribuídas (GADEA; IONESCU; IONESCU, 2020).

3.5.3 CONSISTÊNCIA EVENTUAL FORTE (*SEC*)

Consistência eventual forte define a consistência de réplicas sob o mesmo conjunto de atualizações, mesmo que essas atualizações sejam aplicadas em ordens distintas, desde que a casualidade seja respeitada. Interpretação de operações é, portanto, a base para a compreensão semântica das operações entre as réplicas, pois garante coerência de contexto causal entre as

réplicas. *SEC* assume que a interpretação de operações é correta, isso implica que as semânticas de domínio (especificações de cada sistema) devem ser preservadas para garantir a convergência (GOMES et al., 2017).

Gomes et al. (2017) apresentam exemplos de como a convergência de estado pode ser alcançada por meio de regras que preservam a semântica das operações em cenários concorrentes. Um desses exemplos é intitulado *Observable-Remove Set (OR Set)*, este define remoções de elementos observáveis, isso é: cada elemento pode ser caracterizado por um identificador, a operação de remoção atua apenas sobre elementos nos quais o identificador é conhecido no momento de remoção; caso esses elementos removidos sejam adicionados novamente por uma operação concorrente, essa adição não será eliminada devido a discrepância de identificadores (GOMES et al., 2017). Esta implementação demonstra como a sincronização pode ser alcançada conforme as semânticas de domínio de cada sistema.

Ainda nos exemplos de Gomes et al. (2017), é descrito mecanismo de edição de texto que permite manter a caracteres excluídos, assim réplicas podem usar qualquer caractere como referência para inserir antes ou depois sem precisar considerar ações concorrentes. Isso pode ser aplicado na edição de anotações da aplicação móvel.

Nesse contexto, *CRDTs* garantem consistência eventual forte por especificar comutação de operações concorrentes, assim as réplicas podem convergir sem a necessidade de coordenação entre elas (SHAPIRO et al., 2015).

3.5.4 VETORES DE VERSÃO

Vetores de versão são estruturas lógicas que codificam atualizações em sequências de pares formados pelo local e o número de atualizações daquele local; assim, em vetores de versão, duas réplicas são inconsistentes se, e apenas se, os seus vetores de versão são incomparáveis, isso é, cada réplica foi sujeita a atualizações que a outra não sofreu (PARKER et al., 1983). Preguiça et al. (2010) expande ao afirmar que vetores de versão sintetizam histórias de causalidade ao guardar o prefixo dos eventos gerados em qualquer réplica específica (PREGUIÇA et al., 2010).

A construção histórica dos vetores de versão inicia com os relógios de Lamport, onde é

definido causalidade: se um evento A ocorre antes de um evento B, então o contador de A deve ser menor que o contador de B, contador sendo a marca temporal lógica dos eventos no sistema (LAMPORT, 1978). Relógios de Lamport não podem inferir concorrência, assim, Mattern introduz relógios vetoriais. As posições dos relógios vetoriais são referentes a processos, o valor é referente ao número de eventos ocorridos em cada processo; desse modo, é possível verificar a casualidade e concorrência dos eventos (MATTERN, 1989).

Portanto, relógios de Lamport codificam causalidade, relógios vetoriais detectam concorrência e vetores de versão adaptam o comportamento de relógios vetoriais para sistemas distribuídos.

3.6 TRABALHOS RELACIONADOS

Este trabalho lida com os mecanismos de sincronização de dados, assim trabalhos relacionados a este estão nesse mesmo contexto. Um desses trabalhos é Requisitos para o Projeto de Aplicações Móveis Distribuídas (AUGUSTIN et al., 2001) que apresenta o modelo ISAM para construir um sistema de produção de aplicativos para o contexto móvel; publicado em 2001 permanece relevante até os dias atuais devido o seu trabalho na categorização de contexto. Outro trabalho é *Implementation of Offline-First Architectures for Android Internet-Based Chat Systems* (GUDA, 2025) publicado em 2025 segue a mesma linha, propõe uma estrutura para a criação de um *chat* para redes locais, não lida com redes descentralizadas mas oferece comentários úteis na sincronização de mensagens.

REFERÊNCIAS

ANDROID DEVELOPERS. **Room — Android Jetpack Releases**. [S.l.: s.n.], 2025.

<https://developer.android.com/jetpack/androidx/releases/room>. Última versão estável 2.8.4 em 19 de novembro de 2025. Disponível em:

<https://developer.android.com/jetpack/androidx/releases/room>. Acesso em: 14 dez. 2025.

AUGUSTIN, Iara et al. Requisitos para o projeto de aplicações móveis distribuídas. Versão portuguesa. In: ANAIS do VII Congreso Argentino de Ciencias de la Computación (CACIC). [S.l.: s.n.], out. 2001. Origem: Red de Universidades con Carreras en Informática (RedUNCI). Data de exposição/publicação: outubro 2001.

BAILEY, Tim. **An Introduction to the C Programming Language and Software Design**. [S.l.: s.n.], 2005. Draft 0.6, July 12, 2005. Material didático utilizado em curso de Engenharia de Software.

BIØRN-HANSEN, Andreas; MAJCHRZAK, Tim A.; GRØNLI, Tor-Morten. Progressive Web Apps for the Unified Development of Mobile Applications. In: _____. **Web Information Systems and Technologies (WEBIST 2017)**. Cham: Springer, 2018. v. 322. (Lecture Notes in Business Information Processing), p. 64–86. DOI: 10.1007/978-3-319-93527-0_4.

CADDY SERVER. **Caddy Documentation**. [S.l.: s.n.], 2025.

<https://caddyserver.com/docs/>. Acesso em: 09 dez. 2025.

DOCKER, INC. **Docker Overview**. [S.l.: s.n.], 2025.

<https://docs.docker.com/get-started/docker-overview/>. Documentação oficial — acessado em 2025-12-15. Disponível em:

<https://docs.docker.com/get-started/docker-overview/>. Acesso em: 15 dez. 2025.

DONOVAN, Alan A. A.; KERNIGHAN, Brian W. **The Go Programming Language**. Boston, MA: Addison-Wesley Professional, 2016. First printing October 2015. ISBN 978-0-13-419044-0.

FEYERKE, Alex. **Offline-first: Banco de dados no front-end**. YouTube. 2021. Disponível em: https://www.youtube.com/watch?v=dPz_5-MEvcg. Acesso em: 25 mai. 2025.

FISCHER, Michael J.; LYNCH, Nancy A.; PATERSON, Michael S. Impossibility of Distributed Consensus with One Faulty Process. **Journal of the ACM**, v. 32, n. 2, p. 374–382, 1985.

GADEA, Cristian; IONESCU, Bogdan; IONESCU, Dan. A Control Loop-based Algorithm for Operational Transformation. In: PROCEEDINGS of the IEEE 14th International Symposium on Applied Computational Intelligence and Informatics (SACI 2020). Timișoara, Romania: [s.n.], mai. 2020. P. xx–xx. Accessed: 2025-12-15. Disponível em: <https://www.ncct.uottawa.ca/>. Acesso em: 15 dez. 2025.

GAFFNEY, Kevin P. et al. SQLite: Past, Present, and Future. **Proceedings of the VLDB Endowment**, v. 15, n. 12, p. 3535–3547, ago. 2022. Award ID: 1835446; PAR ID: 10390276. Sponsored by the National Science Foundation. ISSN 2150-8097.

GARCIA-MOLINA, Hector; ULLMAN, Jeffrey D.; WIDOM, Jennifer. **Database Systems: The Complete Book**. Upper Saddle River, New Jersey: Prentice Hall, 2002.

GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 4. ed. São Paulo: Atlas, 2002. ISBN 85-224-3169-8.

_____. **Métodos e técnicas de pesquisa social**. 6. ed. São Paulo: Atlas, 2008.

GILBERT, Seth; LYNCH, Nancy A. The CAP Theorem: Its Implications for Distributed Systems. **Computer**, v. 45, n. 2, p. 30–36, fev. 2012. Published: 03 January 2012. ISSN 0018-9162.

GOMES, Victor B. F. et al. Verifying Strong Eventual Consistency in Distributed Systems. **Proceedings of the ACM on Programming Languages**, ACM, v. 1, OOPSLA, 109:1–109:28, out. 2017. DOI: 10.1145/3133933. Disponível em: <https://doi.org/10.1145/3133933>.

GOODRICH, Michael T.; TAMASSIA, Roberto; MOUNT, David M. **Data Structures and Algorithms in C++**. 2. ed. Hoboken, NJ, USA: John Wiley & Sons, Inc., 2011. ISBN 978-0-470-38327-8.

GUDDA, Varun Reddy. Implementation of Offline-First Architectures for Android Internet-Based Chat Systems. **International Journal of Multidisciplinary Research and Growth Evaluation**, v. 6, n. 3, p. 1200–1203, mar. 2025. ISSN 2582-7138. DOI: 10.54660/IJMRGE.2025.6.3.1200–1203.

KRAKEND. **KrakenD API Gateway Documentation**. [S.l.: s.n.], 2025. <https://www.krakend.io/>. Acesso em: 09 dez. 2025.

LAMPORT, Leslie. Time, clocks, and the ordering of events in a distributed system. **Communications of the ACM**, v. 21, n. 7, p. 558–565, 1978. Disponível em: <https://lamport.azurewebsites.net/pubs/time-clocks.pdf>. Acesso em: 28 jul. 2025.

MATTERN, Friedemann. Virtual Time and Global Clocks in Distributed Systems. In: WORKSHOP on Parallel and Distributed Algorithms. [S.l.: s.n.], 1989. P. 215–226.

NASRALLAH, Ahmed et al. Ultra-Low Latency (ULL) Networks: The IEEE TSN and IETF DetNet Standards and Related 5G ULL Research. **arXiv preprint arXiv:1803.07673**, 2018. Survey comprehensively covers link and network layer latency reduction standards and studies up to July 2018.

OFFLINE FIRST COMMUNITY. **Offline First**. n.d. Disponível em: <https://offlinefirst.org/>. Acesso em: 13 dez. 2025.

PARKER, D. S. et al. Detection of Mutual Inconsistency in Distributed Systems. **IEEE Transactions on Software Engineering**, v. 9, n. 3, p. 240–246, 1983.

PREGUICA, Nuno; BAQUERO, Carlos; SHAPIRO, Marc. Conflict-free Replicated Data Types. **arXiv:1805.00358v1 [cs.DC]**, fev. 2018. Accessed: 2025-12-15. Disponível em: <https://arxiv.org/abs/1805.00358>. Acesso em: 15 dez. 2025.

PREGUIÇA, Nuno et al. **Dotted Version Vectors: Logical Clocks for Optimistic Replication**. [S.l.: s.n.], 2010. arXiv: 1011.5808 [cs.DC].

RESENDE, Kassiano. **Kotlin com Android: Crie aplicativos de maneira fácil e divertida**. São Paulo, SP, Brasil: Casa do Código, 2020.

SHAPIRO, Marc et al. Conflict-Free Replicated Data Types. **CoRR**, abs/1512.00168, 2015. arXiv: 1512.00168 [cs.DC]. Disponível em: <https://arxiv.org/abs/1512.00168>.

SUN, Chengzhang; ELLIS, Clarence (Skip). Operational Transformation in Real-Time Group Editors: Issues, Algorithms, and Achievements. **ACM SIGCHI Bulletin**, v. 30, n. 2, p. 59–66, 1998. Accessed: 2025-12-15. ISSN 1539-297X. Disponível em: <http://www.cs.colorado.edu/eskip/Home.html>. Acesso em: 15 dez. 2025.

TANENBAUM, Andrew S.; BOS, Herbert. **Modern Operating Systems**. 4. ed. Upper Saddle River: Pearson, 2015.

TANENBAUM, Andrew S.; STEEN, Maarten van. **Sistemas distribuídos: princípios e paradigmas**. 2. ed. Upper Saddle River: Pearson Prentice Hall, 2007. Disponível em: <https://csc-knu.github.io/sys-prog/books/Andrew%20S.%20Tanenbaum%20>

%20Distributed%20Systems.%20Principles%20and%20Paradigms.pdf. Acesso em: 20 jul. 2025.

TRAEFIK LABS. **Traefik Proxy Documentation**. [S.l.: s.n.], 2025. <https://traefik.io/>. Acesso em: 09 dez. 2025.

VIOTTI, Paolo; VUKOLIĆ, Marko. Consistency in Non-Transactional Distributed Storage Systems, 2016. arXiv preprint, v4 (12 Apr 2016). arXiv: 1512.00168v4 [cs.DC].

WEIDNER, Matthew. **A brief survey of CRDTs – Part 1**. [S.l.: s.n.], 2023. <https://mattweidner.com/2023/09/26/crdt-survey-1.html>. 26 set. 2023. Acesso em: 20 jul. 2025.